Phase 1 Report – Inception

Jacob Woodard Michelle Messer Scarlett Shropshire

April 3, 2026

Understanding the Problem

As a small team of data scientists working for a major mobile app store, it is crucial to have a clear understanding of the problem we are trying to solve. Our team has been left with a base model that inputs text ratings, and predicts a score. However, many key stakeholders of the company have expressed frustrations with the current system, believing it to be highly inaccurate and too simple. This project is now in the hands of our team, and it is our job to improve it. The current backlog of reviews is over 100,000 across 500 different apps, and completing a model to satisfy the needs of such a large customer base is critical.

Assigning Major Roles

- Scarlett Shropshire – Conducting Research on Tasks, Maintaining Documentation Standards, Architecture Diagram
- Michelle Messer – ML Model Management, Vision Document, Github management, Connecting ML model to website
- Jacob Woodard – Maintaining Project Webpage, Connecting ML model to website, Github management, Vision Document

Although we may have designated roles, Michelle and Jacob share several overlapping responsibilities and will divide these tasks evenly to ensure balanced contributions. All members are expected to maintain detailed documentation to give to other members of the team. This will ensure team members are all on the same page and are knowledgeable on every step of the process.

Though we may have designated roles, all members are expected to maintain detailed documentation to give to other members of the team. This will ensure team members are all on the same page and are knowledgeable on every step of the process.

Stakeholders and Personas

We can make plenty of assumptions of what key stakeholders want out of this product and make improvement decisions based on those assumptions. However, it's best to communicate with them directly to understand their exact concerns and desired outcomes. Due to the size of this company,

there are a plethora of stakeholders that have an interest in this product. From app developers, app store managers, the end-users, platform owners, advertisers, and internal review teams, each stakeholder has different wants and needs when it comes to the use of this project. To get a better understanding of the key motivations and pain points of potential stakeholders, three personas were drafted to gain insight into specific stakeholders.

Morgan Chen: An App Store Moderator / Quality Manager. Morgan's primary job is to moderate the quality of apps across the app store, and she sees the scoring system as a way to simplify her work process. The scoring system allows her to surface problems quickly instead of having to manually read reviews. However she believes that the current model gives unreliable and inconsistent scores, and that the system does not account for biased reviews in the data. She has little trust in the system that she uses daily to streamline her work process.

Maximilian Stein: Mid-Level Mobile App Developer. Maximilian manages performance features for his company, so it is crucial to have accurate and reliable feedback. He wants to utilize the scoring system to track negative reviews and the type of performance issues that are being mentioned. However, he is very frustrated that he has to manually check reviews due to the scoring system inaccurately flagging reviews with bugs mentioned in them as positive. Though Maximilian only uses the scoring system a couple times a week, he heavily relies on the accuracy of results to guide his workflow when developing his performance improvements.

Hailey Irving: App Advertiser for Growing Social Media App. Hailey is head of advertising for EchoSphere, a growing social media app. Hailey wishes to use the scoring system to find the best apps on which to spend her limited advertising budget to run ads on. She also wants to use the system to quickly find which apps to avoid or pull existing ads from. She cannot use the scoring system in its current state, and is frustrated that she has to spend her limited time doing manual research for the best apps to advertise in. An improved scoring system would ideally help her save time and money.

Vision Document (Summary)

Now that we have a better idea of who our stakeholders are, their wants, and their frustrations with the system, we can set a concrete vision for the direction of the system. The stakeholders stressed that their biggest frustration with the system is its unreliable results, and its simple scoring system. The stakeholders rely on this system to streamline their workflow and cut down the time dedicated to research, so it's imperative to have an accurate system. The updated app scoring system will accept unstructured text as input and return a predicted rating for each review, maintaining a minimum accuracy of 80% with a target range of 85–90%. The system will also incorporate filtering functionality to categorize reviews into three bins: positive, neutral, and negative, while sorting reviews on a more granular 1–10 scoring scale. This will give greater complexity to the system, allowing the stakeholders to have more versatility in its utilization.

The updated system will introduce consistency and reliability, empowering stakeholders to make confident, data-driven decisions that ultimately benefit end users. For app store managers, the system provides clearer visibility into which apps are consistently underperforming and in need of further inspection, supporting decisions such as demotion or removal. For app developers, the complaint filtering and trend series features make it easier to pinpoint bugs and prioritize improvements within their apps. For external partners such as advertisers, the system helps identify the most reviewed and top-rated apps, making promotional decisions more targeted and informed. Overall, the updated system addresses the needs of all key stakeholders, delivering consistency, reliability, and efficiency across the board.

Functional Requirements (Summary)

- The system should filter reviews by category, display frequency of complaints, display ratings in ascending order, and detect spam reviews.
- The system should have a scoring system on a 1–10 scale, provide a confidence score on a 0–100% scale, and reflect the content of the review accurately.
- The system should flag apps with scores below 4.0 for inspection and set aside scores with 10.0 for advertising stakeholders.
- The system should flag reviews that contain only a single word, reviews with negative ratings with no additional context, and display a warning indication when confidence scores drop below 60%.

Non-Functional Requirements (Summary)

- The system should have a minimum prediction accuracy of 80% and should flag at least 95% of single word reviews.
- The system should display confidence scores with $\pm 2\%$ precision.
- The system should filter/sort reviews with only 2 interactions, load under 5 seconds, support at least 300 concurrent users, support around 10,000 reviews a week, and retain trend score history for at least 6 months.
- The system should maintain detailed documentation so that a new developer can understand the pipeline within 2 hours.

Phase 2 Report – Elaboration and Planning

Jacob Woodard Michelle Messer Scarlett Shropshire

April 14, 2026

Dataset Analysis

Before preparing strategies for building a new pipeline, it was critical that we had a basic understanding of the data we were working with. We explored the dataset by querying different aspects of it to gain a better understanding of the variables. In this dataset, there are 6 columns: appID, reviewerName, reviewText, reviewerRating, reviewDate, and textAnalytics. In addition, there are 111,143 rows of unique reviews.

For the appID column, there are 492 unique apps with reviews. The app with the largest number of reviews sits at 4,213 reviews, while there are 35 apps with only 1 review. When exploring the reviewerName column, we found that there were around 100,000 unique users submitting reviews. There were not many instances of users submitting repeated reviews, with the most frequent reviewer being "John Smith" with 26 reviews. When investigating the reviewText column, there were a total of 101,000 unique reviews, with approximately 99,000 of the reviews appearing only once in the entire dataset. The remaining reviews in the dataset are simple and easily repeatable strings of text, such as one-word responses like "Good," "Ok," or "Great!" Capitalisation was also heavily affecting the count of unique reviews, so applying some kind of text normalisation in the final pipeline would be valuable.

For the reviewerRating column, we found that the distribution of app ratings is highly skewed toward higher values. For reviewDate, the date format was inconsistent across data points. To fix this inconsistency, we converted everything into a yyyy/mm/dd format. The dates of reviews are sporadic, but the data starts from 2009/02/11 and ends at 2017/10/28. After plotting the counts of reviews by date, there is a steady increasing trend in reviews as the years progress.

Finally, for the textAnalytics column, most of the values are null. There are only 487 rows that have values, ranging from 0.0 to 100.0. Examining the other data in these rows does not give a clear indication of what the column represents. Due to these constraints, it is best to remove this column before feeding data into the pipeline.

Base Pipeline Analysis

Next, it was crucial to gain a basic understanding of how the base pipeline operates so we could understand its processes and limitations. The base pipeline has the following functions: Data

Ingestion, Pre-Processing, Splitting Data, Linear Regression Model, Model Training, and Scoring. We explored each process to understand how it functions and how we can improve the pipeline to align with our requirements.

Data ingestion is the simplest part of the pipeline, feeding the raw data directly into the pipeline without any modifications. The pre-processing step applies many transformations to the dataset, mostly consisting of basic text normalisation. The goal of this process is to standardise input text, reduce noise, and improve consistency for downstream modelling. The major limitation of this step is that nuance in the text is lost during normalisation. Removing special characters, punctuation, exclamation marks, and question marks can impact how the model interprets information, as these features can carry meaningful sentiment weight. The data splitting component is very simple in its current settings, splitting the data only once at 80% into a training set. This can greatly limit accuracy, so there is strong consideration for using a cross-validation method instead.

The current model being used is linear regression, which assumes a straight-line relationship between text features and ratings — an assumption that does not reflect how sentiment actually works. This method appears too simple for the outcome we are trying to produce, so we are considering using a more robust classification model to improve results. In its current state, the scoring process is not producing the results we need. Predictions are returned on the raw 0–1 scale with no rescaling applied, meaning every output is on the wrong scale relative to the 1–10 requirement. Additionally, one of our key requirements — attaching confidence scores to predictions — is not yet in effect, so this feature will need to be built into the pipeline.

Architecture Diagrams

It is important to develop a visual structure and outline of the system, pipeline, and how different components interact with one another. To facilitate this, a context diagram and a component diagram were created to provide a visual understanding of how our system will operate.

For the context diagram, the main system is the app review scoring system, which is where the ML pipeline, 1–10 rating prediction, filtering, content flagging, and confidence scores will all be facilitated. It is important to connect the end users who will be using the system, the external systems that will feed data into it, and the external systems that will display results. The end users consist of our store moderator, app developer, and advertiser stakeholders, all of whom have a vested interest in the system performing according to their stated requirements. The mobile app store and the raw data being fed into the system are key external systems critical to the results of the app review scoring system. Finally, it is important that the app review scoring system connects to an external website to display dashboards. These dashboards will allow end users to navigate trends and scores through filtering systems, enabling them to tailor results to what is most relevant to them.

For the component diagram, it was important that we created a general outline for how we will structure the first iteration of our improved pipeline. We will start with raw data ingestion, followed immediately by pre-processing. In this step, we will perform text normalisation to establish uniform formatting, input validation to account for single-word responses, and sentiment signalling to help preserve the tone of reviews. We will then move on to model training, where we will select our model, perform cross-validation, and train the model to achieve our 80–90% accuracy target. Once complete, we will move to scoring, which will involve attaching confidence scores to the output and implementing logic layers for post-scoring and user interface integration. The final step is connecting the model to an external website with dashboards that can visualise, filter, and rank results.

Risk Register

To prepare for the challenges ahead in Phase 3, we identified and documented the key risks associated with our project. Each risk was assessed based on its likelihood of occurring and its potential impact on our ability to meet the stated requirements.

The most critical risks identified are technical in nature. The current linear regression model does not support confidence score generation, which is a core functional requirement. This will require switching to a classification-based model or building a calibration layer on top of the regression output, and must be resolved in Iteration 1 before any dashboard work can begin. Similarly, the model currently predicts on a 0–1 scale rather than the required 1–10 scale, which must be corrected before any results are presented to stakeholders. Additionally, there is a risk that the model does not reach the 80% minimum accuracy threshold, which we plan to address by exploring alternative models and using cross-validation for a more reliable performance estimate.

Other identified risks include the possibility that spam detection fails to meet the 95% precision and 90% recall requirement, that single-word and empty-negative review flagging misses the 95% coverage threshold, and that persistent output storage is not implemented in time for dashboard development. Class imbalance in the dataset also poses a risk of the model being biased toward predicting the most common rating values regardless of review content.

Drafted Project Page

As part of Phase 2, we created a draft project webpage via GitHub Pages to begin communicating what we are building and why. The page is designed to give any outside reader a clear overview of the project without requiring them to navigate the repository directly.

The current draft includes a project overview describing the purpose and goals of the system, a visual representation of the pipeline stages, a summary of the key functional and non-functional requirements, the two planned iterations for Phase 3, an interactive phase tracker where each project

phase can be expanded to view a summary of completed and upcoming work, and a team section listing all group members. The webpage will continue to be updated throughout Phase 3 and Phase 4 to reflect the current state of the system, experiment findings, and final results.

Phase 3 Report – Construction

Jacob Woodard Michelle Messer Scarlett Shropshire

April 25, 2026

Diagrams

In Phase 2, a considerable amount of time was spent creating context and component diagrams to get a foundational understanding of our system. This was so we could get a basic blueprint and understanding about how our system will operate. Now that work has been done to develop the system and pipeline, it was imperative that we explore other diagrams to show the more complex interactions of components in our system.

The next diagram created was a use case diagram. The purpose of this diagram is to visualize the functional requirements of a system by showing interactions between end-users and the system. Our diagram focused specifically on how our established stakeholders — Store Moderator, App Developer, and Advertiser — interact with the system. Though there are many steps in our process, our end-users will not interact with each step. For example, reviewers will only interact at the stage of feeding information into the raw dataset. However, both the app developer and advertiser will mostly interact with the final deliverables of the pipeline — specifically the external website where trends and dashboards will be displayed. The store moderator will interact with these features as well, but will also have an interest in checking for flagged and spam reviews.

The next diagram created was the class diagram. In general, a class diagram models a system by displaying classes, their attributes, methods, and relationships. These are most commonly used to map code structures in Python, however there is not a considerable amount of script for our model. We decided to create a more general class diagram that modeled an overview of our system components. Significant consideration was given to the drawn arrows of connection for the different classes as well as detailing their multiplicity. For example, the data and pipeline have a multiplicity of one on both sides of their relationship, as there is one dataset and one model in this system. Further, a solid line and open arrow was drawn between the ML pipeline and website as these two classes have direct association with each other.

The final diagram we made is a sequence diagram, which has the purpose of mapping out how objects and components behave, interact, and exchange messages over time to perform a specific function. Fortunately, we have a very straightforward structure to our pipeline, and steps run in a linear fashion, so not much messaging to previous components was necessary to display on the diagram. For the most part, it follows a very clean staircase structure, but there is a deviation when the data is split. This occurs because only 80% of the data is fed to the immediate next step —

upsampling — while the remaining 20% test set is sent directly to the model scoring step.

Pipeline Experiments and Model Development

Phase 3 was focused on actively experimenting with and improving the pipeline established in Phase 2. We ran a series of documented experiments, each with a clear rationale, expected outcome, and recorded results. Our first priority was adding an Evaluate Model component to the baseline so we had a concrete set of metrics to compare against. The baseline Linear Regression model produced an R^2 of only 0.059, confirming it was barely outperforming a naive prediction and giving us a clear starting point.

From there we enabled stratified splitting, added sentiment feature engineering to preserve signals like exclamation marks and review length before preprocessing stripped them, and replaced the Linear Regression model with a Boosted Decision Tree Regression model. Each change produced measurable improvement, bringing R^2 from 0.059 up to 0.151. However the most impactful decision of the phase was reframing the problem entirely. Rather than predicting a continuous numeric rating, we reframed the task as a three-class sentiment classification problem — negative, neutral, and positive — and switched to a Multiclass Decision Forest. This change brought our accuracy to 77.7%, a result far more interpretable and stakeholder-relevant than regression metrics alone could provide.

From there we investigated why our Macro Precision and Macro Recall were low despite strong overall accuracy, and identified class imbalance as the root cause. We added an upsampling script on the training split to balance the three classes, which resolved the bias and brought our final best model to 92.7% overall accuracy with Macro Precision of 0.886 and Macro Recall of 0.901. Each experiment was fully documented in a structured Markdown log committed to the `/experiments/` folder in GitHub.

While there were additional items we had originally planned for this phase — including output rescaling, cross-validation, language detection, and single-word review flagging — time constraints meant we had to prioritize the changes that would have the greatest impact on model performance and system quality.

Webpage Updates

The project webpage was also updated during Phase 3 to reflect the progress made during the construction phase. All five architecture diagrams were added to the page to give outside readers a visual understanding of the system at multiple levels of abstraction. A metrics section was added presenting the results from our best-performing model, giving stakeholders a clear summary of system performance without requiring access to the Azure ML workspace. Minor updates were also made to mark Phase 3 as completed in the interactive phase tracker, and the system pipeline

visualization was adjusted to match the current pipeline structure in Azure ML Studio, replacing the earlier baseline depiction with the updated flow that includes our new components.